

TokHub 通用 Skill 架构与功能说明

这份文档说明新版 `tokhub` skill 的定位、功能范围、运行原理、权限模型和安全边界。它把原来的 `tokhub-admin` 升级为面向所有用户的通用入口，同时保留高权限 `admin-agent` 分支。

版本：0.1.0

日期：2026-07-08

主入口：agent-skills/tokhub

旧名兼容：tokhub 内部处理

1. 一句话定位

新版 `tokhub` skill 是一个本地 agent 到 TokHub 实例的安全连接层。用户下载或启用后，可以在本机终端登录任意本地或远程 TokHub 站点，然后让 agent 按这个账号在网站里已有的权限读取、查询、管理对应的数据。

核心原则：skill 不创建新的权限体系，不绕过网站权限。它只把网站里已经存在的登录态、工作区角色、平台管理员角色和 `admin-agent` token 变成 agent 可调用的本地执行通道。

1 个入口

`tokhub` 面向普通用户和管理员

6 类权限面

`public`、`status-api`、`user`、`console`、`admin-session`、`admin-agent`

83 个 session 操作

覆盖公开页、个人、工作区和部分管理员会话接口

5 类 admin scope

`read`、`write`、`dangerous`、`secrets`、`export`

2. 设计目标

面向所有用户

普通用户可以登录自己的 TokHub 账号，查看自己的工作区、私有通道、Gateway Key、用量、告警、审计和事件数据。

保留管理员能力

平台 owner 或 admin 仍然可以访问后台管理模块。原 `tokhub-admin` 的能力被收进 `admin-agent` 高权限分支。

权限完全等价网站

用户能通过公开页或控制台看到什么，skill 才能看到什么。服务端的 401 和 403 是最终权限结果。

本地凭证不进聊天

密码、Cookie、CSRF、Site Key、Gateway Key、provider key 和 admin-agent token 都只在本机终端或本地文件中处理。

写操作必须留痕

写入、删除、导出、下载、批量、重置、撤销、密钥相关操作都需要 `--execute`、原因和幂等键。

保守的输出策略

所有 JSON 输出递归脱敏。导出和下载必须落文件，默认禁止写入当前 git worktree，避免误提交凭证材料。

3. 组件架构

新版 skill 由一个轻入口、多个参考合同、一个确定性客户端脚本和两套操作目录组成。入口负责触发和边界说明，脚本负责真实执行，参考文件负责在执行前约束 agent 的判断。

组件	位置	作用
Skill 入口	<code>agent-skills/tokhub/SKILL.md</code>	定义何时使用、何时拒绝、执行流程和输出合同。
确定性客户端	<code>scripts/tokhub.mjs</code>	完成登录、profile 管理、请求执行、写操作 guard、admin-agent 分支、脱敏和导出保护。
权限模型	<code>references/permission-model.md</code>	说明六类 auth mode、工作区角色、受保护操作和 secret 处理规则。
登录合同	<code>references/session-auth-contract.md</code>	说明 CSRF、Cookie、profile 文件、本地凭证和 URL 归一化规则。
操作目录	<code>references/operation-catalog.json</code>	列出普通 session 模式可识别的 public、me、console、admin-session 操作。
高权限目录	<code>references/admin-agent-operation-catalog.json</code>	列出 admin-agent 分支下的后台操作、scope、risk 和 guard 要求。
旧名兼容	<code>agent-skills/tokhub</code>	仓库只发布一个 skill。旧 <code>tokhub-admin</code> 名称由 <code>tokhub</code> 的触发边界和 admin-agent 分支承接。

4. 运行原理

skill 的运行可以理解为三层：agent 意图层、本地执行层、TokHub 服务端权限层。agent 负责理解用户要做什么，本地脚本负责以可验证的方式发起请求，TokHub 服务端仍然负责最终鉴权和授权。

Agent 意图层

判断是否是 TokHub 实例操作，读取权限合同和操作目录，确认是否需要登录、写操作授权或 admin-agent 分支。

->

本地执行层

`tokhub.mjs` 处理登录态、CSRF、Cookie、Site Key、Bearer token、输出脱敏和本地文件保护。

->

服务端权限层

TokHub 后端按用户角色、工作区角色、Site Key scope 或 admin-agent scope 返回结果或拒绝。

4.1 普通登录链路

1. 用户提供 TokHub URL、用户名或邮箱，密码只在本机终端输入。
2. 脚本请求 `GET /api/auth/csrf`，拿到 CSRF Cookie 和 token。
3. 脚本提交 `POST /api/auth/login`，得到网站同款 session Cookie。
4. 脚本请求 `GET /api/auth/me` 和 `/api/console/settings`，保存用户摘要和默认工作区。
5. profile 写入 `~/.tokhub/sessions/profile-name.json`，文件权限为 `0600`。

4.2 普通请求链路

读请求直接带 Cookie 访问。写请求会自动带 CSRF。若服务端返回 `csrf_invalid`，脚本只刷新一次 CSRF 并重试同一个请求。

```
node agent-skills/tokhub/scripts/tokhub.mjs request GET /api/console/settings --
profile default
node agent-skills/tokhub/scripts/tokhub.mjs request POST /api/console/gateway-keys \
  --profile default --execute --reason "create workspace key" --idempotency-key
"gwkey-20260708-001"
```

4.3 Open API 状态链路

`/v1/status/*` 不是匿名接口。脚本要求使用 `--site-key-env ENV_VAR`，从本地环境变量读取 Site Key，再通过 `X-Site-Key` 请求服务端。

4.4 admin-agent 链路

owner 可以用浏览器会话等价的账号密码创建短期 admin-agent token。之后机器执行使用 `TOKHUB_BASE_URL` 和 `TOKHUB_ADMIN_AGENT_TOKEN`，不再依赖浏览器 Cookie。

```
node agent-skills/tokhub/scripts/tokhub.mjs admin-agent bootstrap \
  --url https://www.tokhub.me --identifier owner@example.com --save-env
~/.tokhub/admin-agent.env

source ~/.tokhub/admin-agent.env
node agent-skills/tokhub/scripts/tokhub.mjs admin-agent preflight
```

5. 权限模型

模式	接口面	凭证	权限边界
public	<code>/api/public/*</code>	无，或登录 profile	公开页面数据。
status-api	<code>/v1/status/*</code>	本地 Site Key 环境变量	Open API 站点授权范围内的数据。
user	<code>/api/me/*</code>	Cookie + CSRF	当前用户自己的资料、收藏和私有通道上下文。
console	<code>/api/console/*</code>	Cookie + CSRF + workspace	当前工作区成员身份。viewer 可读，operator 可操作，admin 可管理成员和设置。
admin-session	<code>/api/admin/*</code>	Cookie + CSRF + platform role	平台 owner 或 admin 通过网站会话访问后台。
admin-agent	<code>/api/admin/*</code>	Scoped Bearer token	高权限自动化。scope、reason、idempotency 和 audit 共同约束。

权限判断不在 `skill` 里重造。`skill` 可以在本地提前阻止明显越界的请求，例如普通用户访问 `/api/admin/*`，但最终授权仍由 TokHub 服务端完成。服务端返回 403 时，agent 应报告权限不足，不应寻找绕过路径。

6. 功能范围

命令	能力	典型用途
<code>login</code>	建立本地 profile	用户输入 URL、用户名和密码，完成账号授权。
<code>whoami</code>	查看当前用户和工作区	确认账号角色、平台管理员状态和当前 workspace。
<code>preflight</code>	健康检查和权限探测	执行任务前确认目标站点可访问、登录态有效、console 或 admin 面可用。
<code>workspaces list/use</code>	列出和切换工作区	普通用户或企业用户选择要操作的组织空间。

命令	能力	典型用途
<code>request</code>	调用允许的 API 路径	查询公开数据、个人数据、工作区数据，或在有权限时执行后台 session 操作。
<code>admin-agent bootstrap</code>	创建短期 admin-agent token	owner 在本机创建机器执行凭证，保存到本地 env 文件。
<code>admin-agent request</code>	高权限后台自动化	生产健康检查、平台通道、用户、组织、审计、告警和导出等后台任务。
<code>admin-agent audit-verify</code>	审计验证	写操作后按 token id 或 idempotency key 查找 agent 审计记录。

6.1 使用方法

1. 用户先提供 TokHub 站点地址，可以是本地地址，也可以是远程部署地址。
2. 脚本在本机终端完成登录，密码不进入聊天，登录后保存本地 profile。
3. 执行前用 `whoami` 或 `preflight` 确认当前账号、工作区、平台角色和可用权限面。
4. 读操作直接按当前账号权限查询。写操作、导出、下载、删除、批量、重置和凭证相关操作必须带执行确认、原因和幂等键。
5. 管理员自动化统一走 `admin-agent` 分支，旧 `tokhub-admin` 名称只作为兼容语义，不再对应单独 skill 包。

6.2 不同账号能获取的数据

账号或凭证	可获取的数据	常见用途	能带来的帮助
未登录	公开概览、公开通道列表、通道详情、可用率曲线、服务商排行、公开错误摘要、推荐位和站点配置。	了解哪些通道可用，比较服务商稳定性，查看公开状态页。	让 agent 能直接回答公开监控和选型问题，不需要人工打开多个页面截图。
外部站点	<code>/v1/status/*</code> 下的通道、uptime、incident、overview 和 channel sync 数据，范围由 Site Key 绑定的 Open API 站点决定。	同步公开状态页，把 TokHub 监控数据接到第三方站点、看板或客户门户。	把外部展示和内部监控接起来，减少手工导出和重复维护。

账号或凭证	可获取的数据	常见用途	能带来的帮助
普通用户	个人资料、收藏通道、自己有权限进入的工作区入口，以及该账号在网站上能看到的个人数据。	查看收藏、整理常用通道，确认自己能进入哪些组织空间。	帮助用户用自然语言查询自己的 TokHub 数据，同时保持和网站完全一致的权限边界。
查看者	当前工作区设置、网关、Gateway Key 列表及脱敏信息、成员列表、用量、告警、审计、incident 和治理摘要。	查看工作区运行状态、成本趋势、告警历史、审计记录和成员配置。	适合做运营巡检、周报整理、问题追踪和只读审计，不会误改生产配置。
操作员	查看者数据，加上私有通道维护、通道校验、立即探测、网关调试、Gateway Key 创建或撤销、告警规则、通知通道和 incident 处理。	处理上游故障、调整网关入口、创建临时 key、测试告警通道、关闭或重开 incident。	把日常运维动作变成可审计的 agent 操作，减少在后台反复点击和复制参数。
管理员	操作员数据，加上工作区设置、成员管理、批量成员操作和部分重置类动作。	维护组织成员、调整工作区策略、处理权限变更和治理配置。	帮助团队把权限治理、成员变更和配置检查沉淀成可复用流程。
平台管理员	生产健康、平台设置、平台通道、用户、组织、全局用量、后台审计、审计导出、通道导入导出和站点包下载。	平台运营、生产巡检、用户和组织排查、全局成本分析、通道迁移和后台审计。	让管理员能从同一个 skill 完成后台查询和受控变更，降低跨页面操作成本。
机器凭证	由 token scope 限定的 <code>/api/admin/*</code> 后台数据和动作，写操作必须带 reason、idempotency key 和审计验证。	定时巡检、自动导出、CI 检查、生产健康报告、后台批处理和审计回溯。	适合机器自动化场景，既保留高权限能力，也保留 scope、幂等和 audit 约束。

数据不是越权复制。 skill 只是把网站已有权限变成 agent 可调用的通道。明文密钥、Cookie、CSRF、session 文件、Site Key 和 admin-agent token 不进入聊天；导出和下载默认写本地安全路径，并按密钥材料处理。

6.3 应用场景和价值

公开监控问答

查询通道状态、服务商排行、错误摘要和可用率曲线，用于选型、状态页说明和客户答疑。

工作区运维

围绕私有通道、网关、Gateway Key、告警、incident 和审计做日常排查，适合团队值班和问题复盘。

成本和用量分析

读取 usage、rollup、审计和请求事件，整理组织成本、异常用量、重点模型和高频调用来源。

权限和成员治理

查看成员、角色、工作区设置和审计记录，辅助排查谁能访问什么、谁改过什么、哪些权限需要收紧。

平台后台巡检

平台管理员可以检查 production health、全局通道、用户、组织、告警和审计，形成稳定的后台巡检流程。

自动化报告和同步

通过 Site Key 或 admin-agent token 做定时报告、外部状态同步和审计验证，减少人工导出和手动对账。

目的作用：让普通用户、工作区成员和平台管理员都能用同一个 skill 查询和管理 TokHub 数据。它把后台能力变成可审计、可复用、可自动化的本地执行流程，同时不绕过网站权限。

7. 安全机制

路径边界

只允许 `/api/public/*`、`/v1/status/*`、`/api/me/*`、`/api/console/*`、`/api/admin/*`。拒绝 `/gateway/v1/*`、反斜杠、控制字符和 dot segment。

凭证边界

普通登录不读取 `TOKHUB_ADMIN_PASSWORD`。Site Key 只从环境变量读取。admin-agent token 只走本地 env。

CSRF 边界

session 写请求必须带 `X-CSRF-Token`。admin-agent 只在 `/api/admin/*` 上用 Bearer token 绕过 CSRF，由 scope 和 audit 约束。

写操作 guard

非读请求、导出和下载必须带 `--execute`、`--reason`、`--idempotency-key`。

输出保护

JSON 递归脱敏，错误输出脱敏。导出和下载必须 `--output`，不直接打印到终端。

文件保护

profile、env、导出和包文件使用 `0600`。默认不覆盖已有文件，默认不写当前 git worktree。

8. 旧 tokhub-admin 的下线方式

`tokhub-admin` 不再作为独立 skill 包发布，仓库只保留 `tokhub` 一个 skill。系统兼容旧名称时，会把它路由到 `tokhub` 的 admin-agent 分支。

```
node agent-skills/tokhub/scripts/tokhub.mjs admin-agent COMMAND
```

这样可以避免两个 skill 目录和两套客户端逻辑长期分叉，同时让老用户的任务语义继续被新入口接住。

9. 推荐执行流程

1. 先确认任务是否属于 TokHub 实例操作，而不是源代码开发或文档总结。
2. 读取登录合同和权限模型，判断需要 session、Site Key 还是 admin-agent。
3. 用 `login`、`whoami` 或 `preflight` 确认当前身份。
4. 读操作直接执行，写操作先要求用户提供明确意图、原因和幂等键。
5. 导出和下载统一写到本地安全路径，不写仓库，不覆盖已有文件。
6. admin-agent 写操作后运行 `audit-verify`，把审计验证结果作为输出的一部分。

最终效果：普通用户、工作区管理员、平台管理员和机器自动化可以共用一个 skill，但每条链路都有清晰的权限边界和执行守卫。

10. 不做什么

- 不调用 `/gateway/v1/*` 模型推理接口。
- 不帮助普通用户读取 admin-only 数据。
- 不要求用户把密码、Cookie、token、Site Key 或 API key 粘贴到聊天中。
- 不把导出文件、channel-site 包或本地 profile 写进仓库。
- 不把 admin-agent token 链式扩散到 `/api/admin/agent-tokens` 管理接口。